SENIOR SOFTWARE ENGINEER TAKE-HOME ASSIGNMENT

High-Throughput Transaction Import and Reconciliation Service

---

Technology-stack agnostic

Choose the programming language, framework, database, and processing technologies that best demonstrate your engineering judgment.

Candidate Assignment

The objective is to assess how you design, build, test, and reason about a production-oriented backend system.

Background

Build a production-oriented backend service that imports large transaction files, processes them asynchronously, and exposes reconciliation results through an HTTP API.

The assignment is designed to evaluate more than CRUD development. It should demonstrate your understanding of:

- Backend application architecture and programming-language runtime behavior
- Concurrent and asynchronous processing, streaming, and backpressure
- Multi-threading, multiprocessing, or distributed processing
- Dependency injection and separation of concerns
- Database consistency, transaction management, and idempotency
- Concurrency control, error handling, and recovery
- Graceful shutdown, testing, performance optimization, and observability
- Security and technical decision-making

The solution does not need to be feature-complete at enterprise scale. We are more interested in the quality of your design, reasoning, implementation, and trade-offs. Document any features you intentionally leave incomplete.

1. Technology Stack

You are free to choose your preferred technology stack. There are no mandatory programming languages, frameworks, databases, or infrastructure technologies.

Programming Languages

- JavaScript / TypeScript
- Python
- Java
- Go
- C#
- PHP
- Ruby
- Rust
- Other suitable languages

Backend Frameworks

- NestJS, Express, Fastify
- FastAPI, Django
- Spring Boot, ASP.NET Core
- Gin, Laravel, Symfony
- Other well-justified frameworks

Databases

- PostgreSQL, MySQL, SQL Server
- MongoDB, CockroachDB
- Other suitable database technologies

Message Queues and Background Processing

- Redis / BullMQ, RabbitMQ, Apache Kafka
- Celery, Sidekiq, cloud-native queues
- Database-backed job queues or custom worker implementations

Infrastructure and Additional Technologies

- Docker, Docker Compose, Kubernetes, or another suitable deployment approach
- Redis for caching or distributed coordination
- Object storage for uploaded files
- Monitoring and observability platforms
- Worker pools or dedicated processing services
- ORM, query builders, or native database clients

Technology Selection Requirements

- Explain why you selected each major technology.
- Explain the trade-offs of your architectural decisions.
- Avoid unnecessary infrastructure complexity.
- Ensure the project can run locally without paid services or private cloud infrastructure.
- Demonstrate that your selected technologies are appropriate for processing large datasets.

Your choice of technology will not determine your evaluation score. The quality of the implementation and technical decisions will.

2. Business Scenario

A financial platform receives transaction files from multiple providers. A file may contain hundreds of thousands of transaction records.

The platform must:

- Accept transaction files and validate individual records.
- Detect duplicate transactions and calculate a deterministic risk score.
- Produce reconciliation summaries and track processing progress.
- Continue processing valid records when some records are invalid.
- Avoid blocking normal API requests and recover safely from failures.
- Support concurrent imports and maintain data consistency across multiple processing workers.

The system should remain responsive while imports are being processed.

3. Input Format

The service must accept an NDJSON file. Each line contains one transaction:

```
{
  "transactionId": "txn-10001",
  "accountId": "acc-201",
  "merchantId": "merchant-18",
  "amount": 145.75,
  "currency": "USD",
  "timestamp": "2026-07-20T10:25:00.000Z",
  "description": "Subscription payment"
}
```

Field	Type	Requirements
transactionId	string	Required and unique per provider
accountId	string	Required
merchantId	string	Required
amount	number	Greater than zero
currency	string	Three-letter uppercase currency code
timestamp	string	Valid ISO-8601 date
description	string	Optional, maximum 500 characters

A file may contain valid records, invalid JSON, missing fields, duplicate transaction IDs, unsupported currency codes, extremely long strings, records duplicated across separate imports, and records containing unexpected fields.

The application must not terminate the entire import because of one invalid record.

4. Required API

4.1 Create an Import

POST /v1/imports
Content-Type: multipart/form-data
Header: Idempotency-Key: <unique-client-key>

The request must contain one NDJSON file. Expected response: 202 Accepted.

```
{
  "id": "import-id",
  "status": "pending",
  "createdAt": "2026-07-24T08:00:00.000Z"
}
```

- Idempotency-Key is required. Repeating the request with the same key must not create another import.
- Concurrent requests using the same key must not create duplicate imports.
- The uploaded file must not be loaded entirely into memory.
- File-size limits must be enforced and invalid file types rejected.
- The API must return before the complete import has been processed.
- Upload metadata and processing state must be persisted.
- The system must safely handle interrupted uploads.

4.2 Get Import Status

GET /v1/imports/:id

```
{
  "id": "import-id",
  "status": "processing",
  "progress": {
    "processed": 58200,
    "accepted": 57010,
    "rejected": 1190,
    "duplicates": 214
  },
  "startedAt": "2026-07-24T08:01:00.000Z",
  "completedAt": null,
  "failureReason": null
}
```

Supported statuses: pending, processing, completed, failed, cancelling, cancelled. Candidates may introduce additional statuses when justified and documented.

4.3 Cancel an Import

POST /v1/imports/:id/cancel

Cancellation does not need to be immediate, but processing must stop safely at an appropriate checkpoint. A cancelled job must not remain permanently in processing or continue silently writing data after cancellation.

4.4 Get Reconciliation Summary

GET /v1/imports/:id/summary

```
{
  "importId": "import-id",
  "totals": {
    "accepted": 57010,
    "rejected": 1190,
    "duplicates": 214
  },
  "byCurrency": [
    {
      "currency": "USD",
      "transactionCount": 25000,
      "totalAmount": 4350210.42
    }
  ],
  "byRiskLevel": {
    "low": 41000,
    "medium": 14000,
    "high": 2010
  }
}
```

The summary must be based on persisted processing results.

4.5 Get Rejected Records

GET /v1/imports/:id/rejections?limit=50&cursor=<cursor>

```
{
  "items": [
    {
      "lineNumber": 125,
      "reason": "INVALID_CURRENCY",
      "message": "Currency must be a supported three-letter code",
      "rawValue": {
        "transactionId": "txn-100"
      }
    }
  ],
  "nextCursor": "next-cursor"
}
```

- Use cursor-based pagination.
- Do not expose unbounded result sets.
- Limit the amount of malformed raw input that is persisted.
- Avoid returning sensitive internal error information.

4.6 Health Endpoints

Implement separate endpoints: GET /health/live and GET /health/ready. Liveness should verify that the process is alive. Readiness should verify that the service is ready to accept traffic and can access critical dependencies. They should not perform exactly the same checks.

4.7 Metrics Endpoint

Expose GET /metrics. Prometheus format is preferred, but another well-justified monitoring format is acceptable.

5. Transaction Processing

Each transaction must go through the following processing stages.

Step 1: Parsing

Read the file as a stream and parse it line by line. The entire file must never be stored in memory as one string, buffer, array, or object.

- Empty lines
- Invalid JSON
- Final lines without a newline character
- Excessively long lines
- Unicode content
- Stream errors
- Cancellation during parsing

Step 2: Validation

Validate each transaction. Invalid records must be stored as rejected records with line number, machine-readable error code, human-readable message, and limited raw record information. Do not persist extremely large malformed payloads without limits. Validation failures must not trigger infrastructure retries.

Step 3: Normalization

- Trim string values
- Normalize currency codes
- Normalize timestamps
- Normalize descriptions
- Reject dangerous or invalid values
- Ensure deterministic field representation

Normalization rules must be documented and tested.

Step 4: Duplicate Detection

A transaction is considered duplicated when the same transactionId has already been accepted for the same provider. Duplicate detection must work within one file, across separate files, under concurrent imports, across application processes, and after an application restart.

An in-memory Set alone is not sufficient. Use database constraints or an equivalent durable consistency mechanism as part of the correctness design. Clearly define what happens when the same transaction ID is submitted with different transaction content.

Step 5: Fingerprint Calculation

Calculate a deterministic fingerprint using normalized transaction fields. The fingerprint must produce the same result for logically identical transactions. Use a cryptographic hashing implementation available in your selected technology stack.

- Fields included in the fingerprint

- Field ordering
- Normalization rules
- Hash algorithm
- Versioning considerations

Step 6: Risk Score Calculation

Calculate a risk score from 0 to 100 using transaction amount, description length, transaction hour, merchant identifier, and transaction fingerprint.

The exact scoring algorithm is your decision, but it must be deterministic, documented, non-trivial, and testable.

Score	Risk Level
0–39	Low
40–69	Medium
70–100	High

Risk scoring should simulate CPU-intensive processing. It must not significantly block latency-sensitive API requests.

- Worker threads
- Multiprocessing
- A reusable worker pool
- A separate processing service
- A distributed processing architecture
- Another suitable CPU-isolation strategy

Do not create one worker or process for every transaction or HTTP request. Document why your selected approach is appropriate for your programming language and runtime.

Step 7: Persistence

Accepted transactions, rejected records, progress, and summaries must be persisted. Database writes should be batched where reasonable.

- Transaction boundaries
- Partial batch failure
- Duplicate conflicts
- Retry safety
- Progress consistency
- Application crashes between database operations
- Concurrent writes

Step 8: Aggregation

Produce summaries grouped by currency, risk level, merchant, and account. Candidates may calculate summaries during processing or derive them from persisted records. Explain the selected approach and its trade-offs.

6. Performance Requirements

The application should be designed to process files containing at least 500,000 records. Exact execution time is not strictly graded because candidate hardware differs.

- Files are streamed and memory consumption is bounded.
- Processing concurrency is bounded and database writes are batched.
- Backpressure is respected and HTTP endpoints remain responsive during processing.
- CPU-heavy work is isolated from latency-sensitive API execution.
- Failed work does not create unlimited retries.
- Processing does not create an unbounded number of promises, futures, tasks, or queued operations.
- Resource utilization remains controlled under load.

Include a benchmark report showing dataset size, total processing duration, average records processed per second, peak memory consumption, peak heap usage where applicable, CPU utilization, runtime responsiveness or event-loop delay where applicable, worker concurrency, database batch size, machine specifications, and API latency while an import is running.

Include a script or command that generates test files. Example: generate:data --records=500000. The command and implementation may vary according to your selected stack.

7. Runtime and Resource Monitoring

The application must measure and expose relevant runtime health metrics.

- Processing latency, CPU usage, memory usage, and heap usage where applicable
- Active import count and processing queue depth
- Active worker count and database connection utilization
- Request latency and background-processing throughput

For languages and runtimes that support event-loop monitoring, include event-loop delay and event-loop utilization.

In the README, explain how runtime blocking or resource saturation would be detected; what metric thresholds may indicate a problem; how CPU saturation differs from downstream I/O latency; which implementation areas could become bottlenecks; how latency-sensitive endpoints are protected; and how resource utilization is controlled during large imports.

Use monitoring capabilities appropriate to your selected technology.

8. Backpressure and Concurrency

The solution must demonstrate bounded concurrency.

- Maximum simultaneous imports
- Maximum worker tasks
- Maximum database operations
- Maximum queued batches
- Maximum concurrent uploads
- Maximum queued risk-scoring operations

Do not use unbounded processing patterns such as await Promise.all(records.map(processRecord));. Equivalent unbounded task creation in other languages must also be avoided.

Define behavior when the system is at capacity. Possible behaviors include waiting through a bounded queue, rejecting new work with HTTP 429 Too Many Requests, keeping imports in a persistent pending state, applying backpressure to the input stream, temporarily pausing job consumption, or applying per-provider concurrency limits.

Document where concurrency is limited, how limits were selected, what happens when limits are reached, how queue growth is prevented, and how overload is exposed through metrics.

9. Retry Strategy

Implement a retry policy for at least one appropriate operation. It must include maximum attempt count, exponential backoff, jitter, error classification, logging, metrics, and cancellation support.

Do not retry validation failures, duplicate records, permanent database constraint violations, cancelled jobs, or programming errors.

Explain which operations are retryable, why retrying them is safe, how duplicate side effects are prevented, how retry storms are avoided, and what happens after the final attempt fails.

10. Dependency Injection

Dependency injection is required. Core application logic must not directly construct infrastructure dependencies.

- Import repository and transaction repository
- File storage and job queue
- Risk scorer and worker pool
- Clock and ID generator
- Logger and metrics recorder
- Retry policy

Demonstrate clear dependency direction and separation between business logic and technical integrations. Avoid global mutable dependency containers, service locator patterns, importing database clients throughout business logic, hidden singleton state, creating infrastructure dependencies inside use cases, and business logic that can run only inside the selected HTTP framework.

You may use a dependency injection framework, but must demonstrate an understanding of dependency boundaries. At least one unit test must replace a production dependency with an in-memory or fake implementation. The repository should have a clearly identifiable location where concrete dependencies are assembled.

11. Job Reliability

The system must handle process interruption safely. Consider what happens when:

- The API process terminates during upload.
- A processing worker terminates during an import.
- The database connection is lost.
- A process receives a termination signal.
- The same job is delivered twice or a transaction batch is written twice.
- Cancellation is requested during processing.
- Processing succeeds but progress updating fails.
- Progress is updated but the next processing step fails.

At minimum, job state must not remain permanently locked; reprocessing must not duplicate accepted transactions; database writes must be idempotent where appropriate; processing components must support graceful shutdown; in-flight batches must either complete safely or be retried safely; abandoned or stale jobs must be recoverable or clearly marked as failed; and retry attempts must be bounded.

Document the delivery guarantees of your processing design: at-most-once, at-least-once, or effectively-once through idempotent operations and database constraints.

12. Graceful Shutdown

Handle the operating system's termination and interruption signals appropriately.

- Stop accepting new work.
- Mark the service as not ready.
- Stop fetching new jobs.
- Allow an in-flight batch to finish or cancel safely.
- Close HTTP connections.
- Close worker threads, processes, or background consumers.
- Close database connections.
- Exit after a defined maximum grace period.

Document what happens when the grace period expires, how active jobs are recovered, whether the current batch is committed or rolled back, how cancellation differs from shutdown, and how new HTTP traffic is rejected during shutdown.

Avoid terminating the application immediately while work is still active.

13. Database Design

Provide database migrations or an equivalent version-controlled schema-management approach. Your data model will likely need to represent imports, uploaded files, transactions, import-to-transaction relationships, rejected records, processing progress, idempotency keys, and job ownership, leases, or processing attempts.

You do not have to use these exact tables or relationships.

The database design must include primary keys, foreign keys where applicable, unique constraints, useful indexes, import status consistency, duplicate transaction protection, idempotency protection, pagination support, and safe concurrent updates.

Explain important indexes and constraints. Your database must be part of the correctness model, not only a storage mechanism. If you select a NoSQL or distributed database, explain how you achieve equivalent consistency and uniqueness guarantees.

14. Error Handling

Use structured application errors. Errors should have a stable error code, HTTP status when applicable, safe external message, internal diagnostic context, original cause where appropriate, and retryability classification where relevant.

```
{
  "error": {
    "code": "IMPORT_FILE_TOO_LARGE",
    "message": "The uploaded file exceeds the allowed size",
    "requestId": "request-id"
  }
}
```

Do not expose stack traces, database queries, secrets, internal file paths, raw exception objects, or worker implementation details.

Distinguish between request validation errors, record validation errors, business-rule errors, infrastructure failures, retryable failures, programming errors, and cancellation errors.

Unhandled programming errors should not be silently converted into successful or retryable business results.

15. Observability

Implement structured logging. Logs should include useful fields such as request ID, import ID, job ID, worker ID, batch number, processing duration, record count, error code, retry attempt, and shutdown state.

Do not log full uploaded files or sensitive transaction descriptions.

Required metrics should include HTTP request count and duration, import duration, records processed, records rejected, duplicate records, processing failures, queue depth, active processing workers, retry count, CPU utilization, process memory, processing throughput, and database connection utilization.

Runtime-specific metrics should be included where applicable. Tracing is optional but considered a bonus. Avoid using transaction IDs, import IDs, or request IDs as metric labels because they create high-cardinality metrics.

16. Security Expectations

The solution should account for:

- File-size limits and request-body limits
- MIME-type validation and filename sanitization
- Path traversal and malformed NDJSON
- Extremely long lines and unexpected fields
- SQL injection or equivalent database injection risks
- Log injection and denial of service
- Excessive concurrent uploads and sensitive error information
- Temporary-file cleanup and malicious filenames
- Compression bombs if compressed uploads are supported

Do not trust the uploaded filename, client-provided MIME type, number of records claimed by the client, contents of any individual line, client-provided timestamps, or client-provided identifiers.

Authentication is not required for this assignment. However, the implementation should be designed so that authentication and authorization can be introduced without restructuring the entire application.

17. Testing Requirements

Unit Tests

- Transaction validation
- Normalization
- Risk-score calculation
- Import status transitions
- Retry behavior
- Cancellation behavior

- At least one use case using injected fake dependencies

Integration Tests

- Database persistence
- Duplicate constraints
- Idempotency keys
- Import creation
- Concurrent requests with the same idempotency key
- Rejected-record pagination
- Job recovery or redelivery behavior
- Database transaction boundaries

Performance or Load Test

Include one repeatable load or benchmark test demonstrating API responsiveness during an import, memory behavior, processing throughput, runtime behavior, and bounded concurrency.

You may use k6, JMeter, Locust, Artillery, Autocannon, a custom benchmark script, or another suitable load-testing tool. Tests should not depend on execution order or manual database preparation.

18. Required Documentation

README.md

Explain how to run the project, run migrations, generate test data, start all required processes, run tests, run benchmarks, configure environment variables, understand the technology stack and setup, and identify known limitations.

ARCHITECTURE.md

Explain system components, module boundaries, dependency direction, dependency-injection strategy, processing workflow, database consistency strategy, retry strategy, cancellation strategy, graceful-shutdown strategy, duplicate-detection strategy, runtime and performance protection, backpressure strategy, and failure-recovery strategy.

Architecture Diagram

Provide at least one architecture or processing-flow diagram. Mermaid is acceptable.

ADRs

Provide at least three short Architecture Decision Records. Possible subjects include job execution model, worker or process strategy, transaction and idempotency strategy, framework choice, database access strategy, file-storage strategy, and progress-calculation strategy.

BENCHMARK.md

Include benchmark results and interpretation. Do not provide performance numbers without explaining what was measured, how it was measured, what the bottleneck was, whether the result met expectations, and what would be improved next.

19. Docker and Local Development Requirements

Provide a Docker Compose configuration or an equivalent reproducible local development environment. The environment should support a command similar to:

```
docker compose up --build
```

The environment should include the application API, processing worker or equivalent component, database, message broker or queue when used, and Redis when used.

Add health checks, persistent database volume, environment configuration, sensible restart behavior, and service dependency handling. If you choose a different local development or deployment approach, document how evaluators can reproduce the environment. Do not commit real secrets.

20. Development Process

Use Git commits to demonstrate your development process. Good commits should show logical progress, such as:

- Project bootstrap and application boundaries
- Database schema and migrations
- Import creation and file streaming
- Transaction processing
- Idempotency and duplicate handling
- Metrics and graceful shutdown
- Tests and documentation

Avoid submitting the entire implementation in one commit.

A pull request description or SUBMISSION.md should include summary, architecture overview, important decisions, trade-offs, what you would improve with more time, known risks, benchmark summary, and incomplete requirements.

21. Minimum Acceptance Criteria

A submission is considered complete when:

- The project starts successfully.
- The database schema can be created from migrations or equivalent schema setup.
- A valid NDJSON file can be uploaded and the API returns 202 Accepted.
- The import is processed asynchronously without loading the full file into memory.
- Valid transactions are persisted and invalid records are recorded.
- Duplicate transactions are prevented.
- Import progress and summary results can be retrieved.
- Idempotent import creation works.
- The system has clear dependency injection.
- CPU-heavy work does not run directly in the API request handler.
- Graceful shutdown is implemented.
- Important behavior is tested.
- Architecture and performance decisions are documented.

22. Bonus Features

Bonus features are not required. Possible additions:

- OpenTelemetry tracing
- Server-sent events for import progress
- Resumable uploads and processing checkpoints

- Redis-backed queue and dead-letter queue
- Dynamic worker-pool sizing
- Per-provider rate limits
- OpenAPI documentation
- Property-based testing and Testcontainers
- Kubernetes manifests
- File-storage abstraction with local and S3-compatible adapters
- API authentication and authorization
- Import retry endpoint
- Data-retention policy
- Automated performance regression tests

Do not sacrifice required architecture or correctness to implement bonus features.

23. Submission

Submit:

- Git repository URL.
- Publicly accessible deployment URL with a working Swagger/OpenAPI interface or equivalent API documentation.
- The deployed API must be functional and available for endpoint testing during the evaluation period.
- Local setup and execution instructions.
- Pull request or SUBMISSION.md.
- Architecture documentation.
- Benchmark report.

The project should run without access to private services. Any incomplete requirement should be clearly documented.

24. What We Evaluate

- Correctness and programming language/runtime understanding
- Architecture, dependency injection, and separation of concerns
- Database consistency and concurrency control
- Streaming and backpressure
- Error handling and reliability
- Observability and testing quality
- Code readability and documentation
- Security awareness and performance optimization
- Trade-off reasoning and ability to explain and defend technical decisions

We do not evaluate based only on the number of features completed.

The primary objective is to assess how you design, build, test, and reason about a production-oriented backend system.

You are free to select any programming language, framework, database, queue, or infrastructure technology. We encourage candidates to select technologies they are comfortable with while demonstrating sound engineering judgment. We are not looking for a specific technology stack. We are looking for a well-designed, reliable, scalable, maintainable, and thoroughly explained solution.